

Uninformed Search: BFS, DFS, and Uniform-Cost Search

Chapter at a glance

Before you start: Read a graph drawn as nodes and edges, and use a Python list, set, dictionary and for loop.

By the end, you can: state a route-finding task as a search problem; trace BFS, DFS and UCS by hand; explain which quantity each one optimises; read the search code in `utils/core.py`; and judge a search algorithm by completeness, optimality, time and memory.

Lab 1 asks: *how do you fly from Kuala Lumpur to Keflavík without a direct flight?* This chapter introduces three **uninformed** algorithms — Breadth-First Search, Depth-First Search and Uniform-Cost Search — that find routes without geographical guidance. Chapter 3 adds that guidance; Chapter 4 packages and tests the code.

2.1 Route finding as a search problem

A **search problem** has five ingredients, shown here for the airline network built by `utils/openflights.py`.

Ingredient	In general	In Lab 1
State	A situation the search can be in	An airport, named by its IATA code (KUL)
Start	The initial state	The origin airport of a query
Goal	The state we want to reach	The destination airport
Successors	The states reachable in one step	The airports with a direct route from here
Path cost	The accumulated cost of a path	Number of legs, or the sum of great-circle distances in km

The cost definition matters: BFS and DFS treat each route as one step; UCS sums kilometres. “Shortest route” is ambiguous until the cost is stated.

In Python the graph is a dictionary from each airport to a list of (`neighbour`, `distance_km`) pairs:

```
graph["KUL"][:3]
# [('ADL', 5681.8), ('AKL', 8703.4), ('ALA', 5130.5)]
```

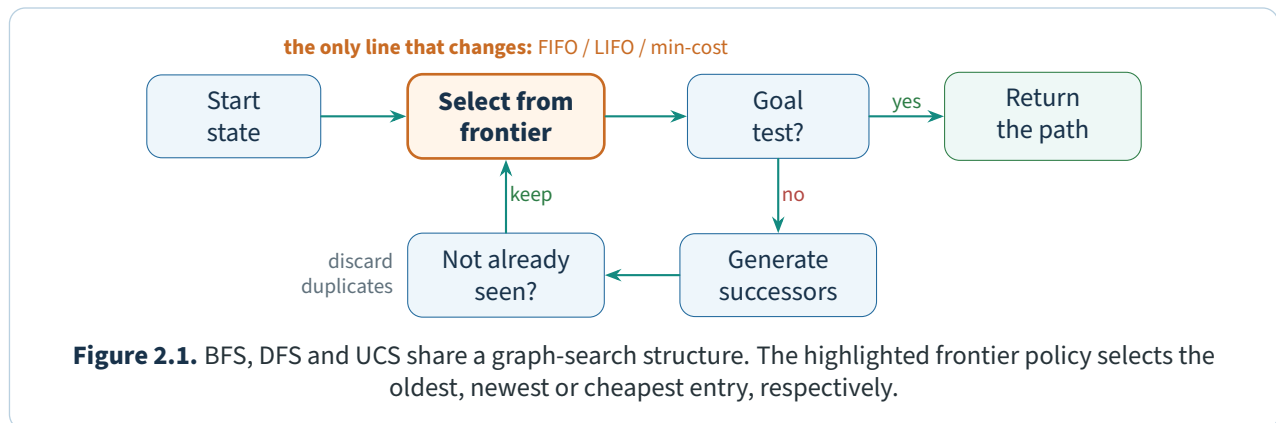
The cleaned OpenFlights graph has 6,464 airports and 37,337 directed routes. Distance is an educational proxy for fares, schedules and connection costs.

Conceptual intuition

The problem supplies successors; the search algorithm chooses which frontier state to examine next. Airport details belong to the first, the distinction between BFS, DFS and UCS to the second.

2.2 The graph-search loop

All three algorithms maintain a **frontier** of discovered but unexamined states and track visited states to avoid repeated work. Their frontier policies differ (Figure 2.1).



Two bookkeeping choices are shared by every implementation in `utils/core.py`:

Parent links instead of copied paths. Record each state's predecessor, then reconstruct the path backwards from the goal. This saves storing a full path in every frontier entry.

Effort counters. Count states **expanded** (whose successors are generated) and peak frontier size. Unlike runtime, these measures are machine-independent.

The BFS function is short enough to read in full; DFS and UCS differ from it in a handful of lines each.

```
def bfs(graph, start, goal):
    started = perf_counter()
    frontier = deque([start])           # FIFO queue
    parent = {start: None}              # also serves as the "seen" set
    expanded, max_frontier = 0, 1
    while frontier:
        state = frontier.popleft()      # oldest entry first
        if state == goal:
            return _result(True, parent, goal, graph, expanded, max_frontier, started)
        expanded += 1
        for neighbour, _ in graph.get(state, {}):
            if neighbour not in parent:
                parent[neighbour] = state
                frontier.append(neighbour)
        max_frontier = max(max_frontier, len(frontier))
    return _result(False, parent, goal, graph, expanded, max_frontier, started)
```

Common mistake

Mark a state as seen when it is *added* to the frontier, not when it is removed. The parent dictionary does this: a state gets a parent the moment it is discovered. Marking on removal lets the same airport be queued many times by different neighbours, which inflates the frontier and, for BFS, can attach a longer path than necessary.

2.3 Breadth-First Search

Breadth-First Search (BFS) uses a first-in, first-out queue, examining states in increasing numbers of steps from the start. Its first path to the goal therefore has the fewest edges.

Step 1: Put the start state in the queue.

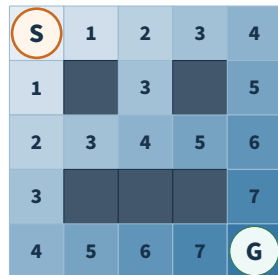
Step 2: Remove the *oldest* state from the front of the queue.

Step 3: If it is the goal, rebuild the path from the parent links and stop.

Step 4: Add each successor that has not been seen to the back of the queue, recording its parent.

Step 5: Repeat until the queue is empty.

BFS expands as a wavefront (Figure 2.2): it must pass distance 7 before reaching distance 8.



Each number is the step at which BFS first reaches that cell.

The goal is found at distance 8, and no shorter route exists: BFS could not have reached it earlier without skipping a ring.

On the airline graph the “rings” are airports one leg away, two legs away, and so on.

Figure 2.2. Cells are shaded by their first BFS arrival step; dark cells are walls. Layer-by-layer expansion reaches the goal in the fewest steps.

Worked example

BFS from KUL to KEF expands 284 airports and returns KUL → AMS → KEF: two legs, 12,274 km. This minimises legs, not kilometres; BFS ignores distances.

2.4 Depth-First Search

Depth-First Search (DFS) uses a stack to examine the newest state first. It follows one branch until exhausted, then backtracks (Figure 2.3).

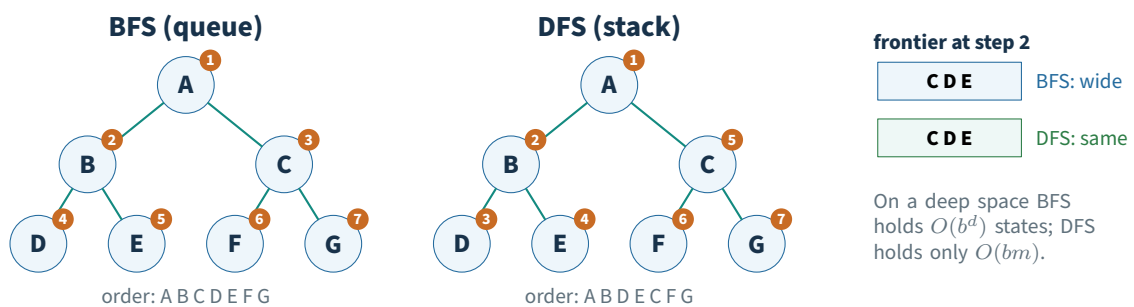


Figure 2.3. Numbered visit orders on one tree. BFS completes each level; DFS exhausts B's subtree before C. The frontier illustrates their different memory demands.

The code differs from BFS in two lines: `frontier.pop()` takes the newest entry, and the successors are pushed in *reverse* order so that the first-listed neighbour is popped first. That second line is why DFS is **order dependent**: it commits to whichever neighbour the graph happens to list first and only reconsiders when that whole branch fails.

Worked example

From KUL, DFS commits to the alphabetically first neighbour, ADL (Adelaide), then to Adelaide's first neighbour, and so on. It reaches Keflavík after 23 legs and 41,768 km — more than the circumference of the Earth. Reversing every neighbour list, so that the last-listed neighbour is tried first, gives a different route of 8 legs. Neither is a meaningful travel plan: DFS answers “is there *a* route?”, not “what is a good one?”.

Conceptual intuition

BFS expands a wavefront; DFS follows one thread. The wavefront is expensive to store but arrives by the fewest steps; the thread is cheap to store but says nothing about how good its path is.

2.5 Uniform-Cost Search

Uniform-Cost Search (UCS) is the algorithm to use when edges have different costs. It keeps the frontier in a **priority queue** ordered by $g(n)$, the cost of the cheapest known path from the start to state n , and always expands the cheapest frontier state. When the goal is expanded, the path to it is the cheapest of all, provided no edge cost is negative.

Figure 2.4 shows BFS and UCS disagreeing on a small weighted graph. Both routes use three edges, so BFS has no reason to prefer either; UCS adds the weights and finds that one costs twice the other.

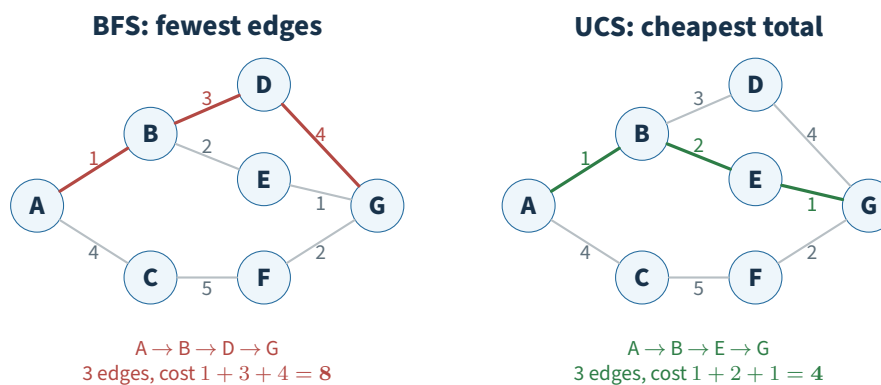


Figure 2.4. The same weighted graph solved twice. BFS returns the first three-edge path it meets, costing 8. UCS orders its frontier by accumulated cost $g(n)$ and returns the three-edge path costing 4. Neither is wrong; they answer different questions.

Worked example

Trace UCS on the graph of Figure 2.4. The frontier holds (g, state) pairs and always pops the smallest g .

expand	frontier after expansion
A (0)	(1, B), (4, C)
B (1)	(3, E), (4, C), (4, D)
E (3)	(4, C), (4, D), (4, G)
C (4)	(4, D), (4, G), (9, F)
D (4)	(4, G), (9, F) (<i>G</i> via <i>D</i> would cost 8: not pushed)
G (4)	goal popped: return $A \rightarrow B \rightarrow E \rightarrow G$ at cost 4

When D is expanded it offers a route to G costing $4 + 4 = 8$, which is worse than the known cost of 4, so nothing is pushed. Equal-cost ties (C , D , G all at 4) are broken by insertion order, which does not affect the minimum cost.

In `utils/core.py`, UCS, GBFS and A* share one function, `best_first`, because they differ only in the priority they attach to a frontier entry. The frontier is a Python `heapq` of tuples (`priority`, `counter`, `state`, `g`):

```
_, _, state, g = heappop(frontier)
if state in expanded_states:
    continue  # an older, costlier entry for this state
if state == goal:
    return ...
expanded_states.add(state)
for neighbour, cost in graph.get(state, {}):
    new_g = g + cost
    if neighbour not in expanded_states and new_g < best_g.get(neighbour, inf):
        best_g[neighbour] = new_g
        parent[neighbour] = state
        counter += 1
        heappush(frontier, (priority(neighbour, new_g), counter, neighbour, new_g))
```

Test for the goal when a state is *popped*, not generated, to avoid returning the first rather than cheapest path. Cheaper paths may add duplicate entries; `expanded_states` skips stale copies. The `counter` breaks priority ties by insertion order without comparing potentially incompatible state types (Chapter 4).

Key idea

BFS orders the frontier by discovery time, DFS by recency, UCS by accumulated cost. The graph, the successor function and the goal test never change; only the frontier policy does, and that single choice is the name of the algorithm.

2.6 Comparing the three methods

Method	Frontier rule	Data structure	What it optimises
BFS	Oldest first	FIFO queue (deque)	Fewest edges; every route counts as one leg
DFS	Newest first	Stack (list)	Nothing — finds <i>a</i> path, order dependent
UCS	Lowest $g(n)$	Min-heap (heapq)	Cheapest total cost, e.g. kilometres

Search algorithms are judged on four standard properties. Write b for the **branching factor** (successors per state), d for the depth of the shallowest goal, and m for the maximum depth of the space.

Completeness If a solution exists, is it guaranteed to be found?

Optimality Is the solution returned guaranteed to be the cheapest?

Time How many states may be expanded?

Space How many states may be held in memory at once?

Method	Complete?	Optimal?	Time	Space
BFS	Yes, if b is finite	Only when every step costs the same	$O(b^d)$	$O(b^d)$
DFS	Not in general	No	$O(b^m)$	$O(bm)$
UCS	Yes, if step costs $\geq \varepsilon > 0$	Yes, for non-negative costs	$O(b^{1+\lceil C^*/\varepsilon \rceil})$	same as time

Here C^* is the cost of the optimal solution and ε the smallest step cost. Figure 2.5 shows what $O(b^d)$ means in practice: at branching factor 3 a goal eight levels down leaves about 9,800 states in memory; at branching factor 5 the same depth needs nearly 500,000. The airline graph has a mean branching factor near 6 but its hubs have over a hundred routes each, which is why the maximum frontier in Lab 1 reaches 1,679 airports for a two-leg BFS answer.

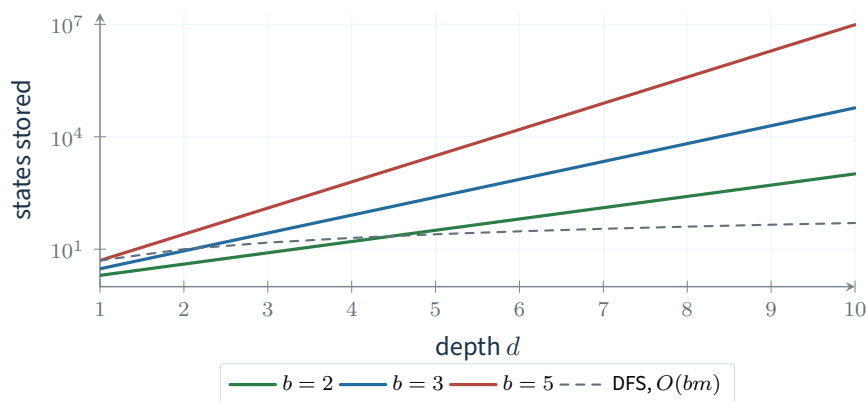


Figure 2.5. States a BFS frontier may hold at depth d , for three branching factors. The vertical axis is logarithmic, so each straight line is exponential growth. DFS stores $O(bm)$ instead, which on these scales is a flat line near the bottom.

Common mistake

Two qualifications matter:

“BFS is optimal” requires *equal* step costs. BFS returns KUL → AMS → KEF at 12,274 km; UCS returns KUL → HKT → LED → HEL → KEF at 11,461 km. Fewest legs need not mean fewest kilometres.

“DFS is not complete” concerns infinite spaces or cycles without a seen-set. On the finite lab graphs, the parent dictionary ensures termination and finding a reachable goal, but not an optimal path.

2.7 What the lab measures

Lab 1 runs each algorithm on three queries and records the following fields.

Field	What it tells you
hops, path_cost_km	The two objectives. Compare BFS on the first, UCS on the second.
expanded	States whose successors were generated: the machine-independent measure of effort.
max_frontier	Peak memory demand in states; the space column of the table above, measured.
runtime_ms	Wall-clock time. Useful within one machine, meaningless across machines.
path	The evidence. Every leg must exist in the graph and the legs must sum to the cost.

Validate results independently: Lab 1 checks every returned leg against the graph and recomputes its cost from the edge list, catching impossible paths before reporting them.

Worked example

The three queries of Lab 1, as the notebook reports them:

query	BFS		DFS		UCS	
	hops	km	hops	km	hops	km
KUL → KEF	2	12,274	23	41,768	4	11,461
SIN → EZE	2	21,976	39	65,214	3	17,817
NRT → CPT	2	19,006	64	91,541	3	14,813

BFS minimises hops, UCS kilometres, and DFS neither. UCS uses one or two extra legs to save 800–4,200 km.